



PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements
for the Award of the Degree of

Bachelor of Technology
in
Computer Science and Engineering

Project Title:

***"AQI-Pulse: Cloud-Based Air Quality Dashboard
with DevOps"***

Submitted by:

Archi Dubey

Registration No.: 12321605

Section: 4OM56

Roll no.: 41

Batch: 2023–2027

Under the Supervision of:

Ms. Arshiya Pathania

GITHUB LINK

<https://github.com/ARCHI-DUBEY/aqi>

CERTIFICATE

This is to certify that the Project Report titled "AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps" has been submitted by Archi Dubey (Reg. No.: 12321605) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Lovely Professional University, Phagwara, Punjab.

The project work embodies an original contribution by the student to the best of our knowledge and has not been submitted elsewhere for the award of any degree or diploma. The student has satisfactorily completed all the requirements for the project as laid down for the degree.

The project, *AQI-Pulse*, is a cloud-native air quality monitoring platform developed using modern DevOps technologies including Docker, Kubernetes (K3s), Jenkins, AWS EC2, Prometheus, and Grafana. The system provides real-time Air Quality Index monitoring, containerized deployment, automated CI/CD workflow, monitoring, and cloud-based orchestration. The implementation demonstrates practical exposure to cloud-native system design and deployment methodologies.

Internal Supervisor:
Arshiya Pathania
Department of Computer Science and Engineering,
Lovely Professional University

Date: 27/05/2026

Place: Phagwara, Punjab

DECLARATION

I hereby declare that the project report entitled "AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps", submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Lovely Professional University, Phagwara, is an authentic record of work carried out by me under the guidance of Ms. Arshiya Pathania during the academic session 2025–2026.

I further declare that the work presented in this project report is original and has been carried out by me independently. The content included in this report has not been submitted previously, either wholly or partially, to any other institution or university for the award of any degree, diploma or certification.

I understand that if any information provided in this report is found to be incorrect or misleading at any stage, the university has the right to take appropriate action as per academic regulations.

The references and sources used during the development of this project have been properly acknowledged wherever required.

Archi Dubey
Registration Number: 12321605

Date: 27/05/2026

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude and appreciation to all those who contributed directly or indirectly towards the successful completion of this project titled "AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps".

I would like to express my heartfelt gratitude to Lovely Professional University for providing an excellent academic environment, infrastructure and resources that supported the completion of this project.

I extend my deepest thanks to my project guide, Ms. Arshiya Pathania, for her continuous guidance, encouragement and valuable suggestions throughout the development of this project. Her support and mentorship helped me understand both the technical and practical aspects involved in designing and implementing a cloud-native system.

I also express my gratitude to all faculty members of the Department of Computer Science and Engineering for providing knowledge and support during my academic journey.

I would like to thank my friends and classmates for their suggestions, cooperation and motivation throughout various stages of project development.

Finally, I would like to acknowledge the contribution of open-source technologies and communities including Python, Streamlit, Docker, Jenkins, Kubernetes, AWS, Prometheus and Grafana, which played a significant role in successful implementation of the project.

I sincerely thank everyone who directly or indirectly contributed toward completion of this work.

Archi Dubey

Registration Number: 12321605

Department of Computer Science and Engineering

Lovely Professional University

ABSTRACT

Air pollution has become a major environmental and public health concern due to increasing industrialization, urbanization and vehicle emissions. Poor air quality can negatively affect human health, climate conditions and overall quality of life. Monitoring Air Quality Index (AQI) and providing users with meaningful information has therefore become essential. Traditional air quality applications often focus only on displaying data and generally lack scalability, deployment automation, cloud integration and monitoring capabilities.

The proposed project, "AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps", is designed as a cloud-native solution that integrates AQI prediction, visualization and modern DevOps practices into a single platform. The project aims to provide users with an interactive dashboard for monitoring air quality while demonstrating real-world implementation of cloud technologies and deployment workflows.

The application is developed using Python and Streamlit, where Streamlit provides an interactive user interface and Python handles application logic and prediction functionality. A machine learning model is integrated for AQI-related prediction tasks. To ensure portability and eliminate dependency-related issues, the application is containerized using Docker.

For deployment automation, a Jenkins CI/CD pipeline is implemented. Jenkins automates processes including source code retrieval from GitHub, Docker image generation, image publishing and Kubernetes deployment. The Docker image is maintained using Docker Hub as a centralized image registry.

The project is deployed on an AWS EC2 m7i-flex.large instance configured with a single-node Kubernetes K3s cluster. Kubernetes manages deployment using pods, services and replica-based architecture while also providing self-healing functionality.

For monitoring and observability, Prometheus continuously collects application metrics and Grafana provides visual dashboards for system monitoring and performance analysis.

This project demonstrates practical implementation of modern DevOps concepts including containerization, orchestration, CI/CD automation and cloud-native deployment practices.

Keywords: AQI, DevOps, Docker, Jenkins, Kubernetes, AWS EC2, Prometheus, Grafana, Cloud Computing, Monitoring System

TABLE OF CONTENTS

Chapter 1: Introduction	10
1.1 Background	10
1.2 Problem Statement	10
1.3 Objectives	11
1.4 Scope of Project	11
1.5 Motivation	12
Chapter 2: Literature Survey	13
2.1 Existing Systems	13
2.1.1 Traditional AQI Monitoring Systems	13
2.1.2 Dashboard-Based Monitoring Platforms ...	13
2.1.3 Cloud-Based Monitoring Systems	14
2.2 Limitations of Existing Systems	14
2.3 Need for Proposed System	15
2.4 Comparative Analysis of Existing Systems ...	16
Chapter 3: Proposed System	17
3.1 System Overview	17
3.2 Key Features	18
3.3 Functional Requirements	18
3.3.1 User Requirements	19
3.3.2 Infrastructure Requirements	20
3.4 Non-Functional Requirements	21
3.5 High-Level Workflow	21
Chapter 4: System Architecture	23
4.1 Overall System Architecture	23
4.2 Frontend Architecture	24
4.3 Backend Architecture	24
4.4 Deployment Architecture	25
4.5 Monitoring Architecture	27

Chapter 5: Technology Stack	28
5.1 Technology Overview and Justification	28
5.2 Python and Streamlit	28
5.3 Docker	29
5.4 Kubernetes (K3s)	30
5.5 Jenkins	31
5.6 Prometheus and Grafana	33
5.7 AWS EC2	34
Chapter 6: Containerization and Deployment ...	35
6.1 What is Containerization	35
6.2 Docker Architecture	35
6.3 Port Configuration	37
6.4 Kubernetes Deployment	38
6.5 Node-Port Configuration	39
Chapter 7: CI/CD Pipeline using Jenkins	40
7.1 What is CI/CD	40
7.2 Jenkins Pipeline Architecture	41
7.3 Pipeline Stages	42
Chapter 8: Kubernetes Orchestration	44
8.1 What is Kubernetes	44
8.2 Why Kubernetes (K3s)	44
8.3 Deployment Configuration	45
8.4 Node-Port Service	47
Chapter 9: Monitoring and Observability	49
9.1 Observability Principles	49
9.2 Prometheus Metrics Collection	49
9.3 Grafana Dashboard	51
9.4 Monitoring Workflow	53
Chapter 10: AWS Cloud Deployment	54
10.1 EC2 Infrastructure	54
10.2 Security Group Configuration	54
10.3 Deployment Workflow	55

Chapter 11: Testing and Validation	56
11.1 Functional Testing	56
11.2 Kubernetes Validation	56
11.3 Monitoring Validation	57
11.4 Deployment Validation	58
Chapter 12: Challenges Faced and Solutions ...	59
12.1 Jenkins Issues	59
12.2 Kubernetes Issues	59
12.3 Monitoring Issues	60
Chapter 13: Results and Discussion	61
Chapter 14: Advantages of System	63
Chapter 15: Limitations	64
Chapter 16: Future Scope	65
Chapter 17: Conclusion	66
Chapter 18: References	67
Appendix	68
A. Important Commands Reference	68

Chapter 1: Introduction

1.1 Background

Air pollution has become one of the major environmental issues affecting human health and quality of life across the world. Rapid industrialization, increasing urban population and rising vehicle emissions have significantly contributed to deterioration in air quality levels. Exposure to polluted air can cause several health problems including respiratory diseases, cardiovascular disorders and other long-term medical conditions. Therefore, continuous monitoring and analysis of air quality have become important in modern society.

Air quality is generally represented using the Air Quality Index (AQI), which indicates pollution levels and helps users understand environmental conditions through standardized categories such as Good, Moderate, Poor and Hazardous. AQI information enables people to take preventive measures and make informed decisions regarding outdoor activities and health safety.

Traditional air quality systems mainly focus on displaying information and often lack scalability, deployment automation and monitoring capabilities. Many applications also suffer from dependency issues and manual deployment processes, making maintenance difficult. Modern applications therefore increasingly adopt cloud technologies and DevOps practices to improve portability, reliability and operational efficiency.

The project AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps combines air quality monitoring with cloud-native deployment practices. The system integrates application development, containerization, CI/CD automation, orchestration and monitoring into a unified platform. Technologies including Python, Streamlit, Docker, Jenkins, Kubernetes, AWS, Prometheus and Grafana are integrated to create a scalable and cloud-based deployment architecture.

The project demonstrates how DevOps practices can be applied to develop and deploy a modern application while improving automation, monitoring and deployment efficiency.

1.2 Problem Statement

Traditional air quality applications face several limitations in terms of deployment, scalability and monitoring. Manual deployment processes often create dependency conflicts and increase maintenance complexity. Most systems

do not provide automated deployment pipelines and lack proper monitoring mechanisms.

Additionally, when application failures occur, recovery generally requires manual intervention, leading to service interruptions and reduced reliability. There is therefore a need for a cloud-native platform capable of providing portability, automation, scalability and observability.

The proposed system aims to address these challenges through Docker-based containerization, Jenkins automation, Kubernetes orchestration and monitoring using Prometheus and Grafana.

1.3 Objectives

The objectives of the proposed system are:

- To develop an interactive AQI dashboard using Python and Streamlit.
- To provide users with air quality monitoring and prediction features.
- To containerize the application using Docker.
- To automate deployment using Jenkins CI/CD.
- To deploy the application on AWS EC2.
- To orchestrate workloads using Kubernetes K3s.
- To implement monitoring using Prometheus and Grafana.
- To demonstrate cloud-native deployment architecture.

1.4 Scope of Project

The project scope includes development of an AQI dashboard along with implementation of DevOps technologies and cloud deployment mechanisms. The system covers application development, Docker containerization, Jenkins pipeline automation, Kubernetes deployment and monitoring architecture.

The project currently uses a single-node Kubernetes cluster and focuses on deployment automation and observability. Advanced features such as multi-node

clusters, autoscaling and alerting mechanisms are considered outside the present scope.

1.5 Motivation

The motivation behind this project is to understand real-world implementation of DevOps technologies and cloud-native deployment architectures. Modern organizations increasingly use Docker, Kubernetes, Jenkins and cloud platforms for building scalable systems.

This project provides practical exposure to deployment workflows, automation pipelines and monitoring tools while addressing an important environmental problem related to air quality monitoring. It bridges the gap between theoretical learning and industry practices by combining software development with deployment engineering.

Chapter 2: Literature Survey

2.1 Existing Systems

Air quality monitoring systems are widely used to analyze pollution levels and provide environmental information to users. Over the years, different platforms and applications have been developed to monitor air quality and display Air Quality Index (AQI) information. These systems differ based on their architecture, deployment methods, infrastructure and monitoring capabilities.

Traditional systems mainly focus on collecting and presenting environmental data but often lack automation and scalability features. Recent systems increasingly use cloud technologies and dashboard-based approaches to improve accessibility and visualization. However, many existing applications still face limitations related to deployment complexity, monitoring and maintenance.

To understand the need for the proposed system, it is important to study currently available solutions and identify their strengths and limitations.

2.1.1 Traditional AQI Monitoring Systems

Traditional AQI monitoring systems primarily focus on collecting pollution data from sensors and displaying Air Quality Index information through websites or standalone applications. These systems generally monitor parameters such as particulate matter, carbon monoxide, sulfur dioxide and other pollutants to determine environmental conditions.

Most traditional systems follow a basic architecture where data collection and display functionalities are tightly connected. Deployment and maintenance are usually performed manually, which increases operational complexity and creates dependency issues across different environments.

Although these systems provide essential AQI information, they often lack scalability, automation and monitoring capabilities. In many cases, system failures require manual intervention, making maintenance difficult and reducing overall reliability.

2.1.2 Dashboard-Based Monitoring Platforms

Dashboard-based monitoring platforms provide a more interactive approach for visualizing environmental and analytical data. These systems present information

through charts, graphs, tables and real-time visual components, allowing users to understand trends more effectively.

Modern dashboard platforms improve user experience by offering better visualization and accessibility compared to traditional systems. Users can monitor AQI values, pollution statistics and historical data through a centralized interface.

However, many dashboard systems mainly focus on frontend visualization and often do not provide deployment automation, cloud integration or infrastructure monitoring capabilities. Managing application health and system performance therefore becomes difficult in large-scale environments.

2.1.3 Cloud-Based Monitoring Systems

Cloud-based monitoring systems utilize cloud infrastructure to host applications and manage services through internet-based platforms. These systems provide improved accessibility, scalability and resource management compared to traditional deployment approaches.

Modern cloud systems increasingly use technologies such as containerization and orchestration to simplify deployment and maintenance processes. Cloud platforms allow applications to be deployed on virtual servers and accessed remotely without requiring dedicated local infrastructure.

Although cloud-based systems improve deployment flexibility and scalability, many existing implementations still focus only on hosting applications and lack integrated monitoring, CI/CD automation and complete observability mechanisms. These limitations create challenges in system management and operational efficiency.

2.2 Limitations of Existing Systems

After studying existing air quality and dashboard-based systems, several limitations were identified. Most traditional applications focus only on data presentation and do not provide modern deployment or infrastructure management capabilities.

The major limitations observed are:

- Manual deployment and configuration processes

- Dependency conflicts across different environments
- Lack of containerization support
- Limited scalability capabilities
- Absence of CI/CD automation
- Lack of real-time monitoring mechanisms
- Difficult maintenance and recovery procedures
- No self-healing capability
- Limited observability and performance tracking

These limitations reduce system reliability and increase operational complexity, especially when applications need to support cloud deployment and continuous updates.

2.3 Need for Proposed System

The limitations observed in existing systems create the need for a modern platform that combines air quality monitoring with cloud-native deployment practices and DevOps automation. Current systems often provide AQI information but lack scalability, deployment automation and monitoring capabilities.

There is a requirement for a system that can provide portability, reliability and simplified deployment while maintaining better operational visibility. Integrating modern technologies can improve system performance and reduce manual management efforts.

The proposed system, AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps, is designed to address these challenges by integrating Docker for containerization, Jenkins for CI/CD automation, Kubernetes for orchestration, AWS EC2 for cloud deployment and Prometheus-Grafana for monitoring.

The system aims to provide a scalable and manageable architecture that supports automation, self-healing and real-time monitoring capabilities.

2.4 Comparative Analysis of Existing Systems

The following table presents a comparison between existing systems and the proposed **AQI-Pulse** platform based on major technical characteristics.

Feature	Traditional Systems	AQI Dashboard Platforms	Proposed AQI-Pulse System
Docker Support	No	Partial	Yes
Kubernetes Support	No	No	Yes
CI/CD Integration	No	Limited	Yes
Cloud Deployment	Limited	Partial	AWS EC2
Monitoring	Basic	Partial	Prometheus Grafana +
Scalability	Low	Medium	High
Self-Healing	No	No	Yes
Automation	Manual	Partial	Jenkins Pipeline
Architecture	Monolithic	Dashboard-based	Cloud Native

From the comparative analysis, it can be observed that the proposed AQI-Pulse system provides better scalability, automation, monitoring and deployment flexibility compared to existing solutions.

Chapter 3: Proposed System

3.1 System Overview

The proposed system, AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps, is designed as a cloud-native platform that combines air quality monitoring with modern DevOps technologies. The system provides users with an interactive dashboard for accessing AQI-related information while implementing automated deployment, containerization and monitoring capabilities.

The application is developed using Python and Streamlit, where Streamlit provides an interactive user interface and Python handles application functionality. The application is containerized using Docker to ensure portability and consistency across environments.

A Jenkins CI/CD pipeline is integrated to automate software build and deployment activities. The Docker image generated through Jenkins is stored in Docker Hub and deployed on an AWS EC2 instance configured with a single-node Kubernetes K3s cluster.

For monitoring and observability, Prometheus collects application metrics and Grafana visualizes the collected data through dashboards. The complete architecture creates an automated workflow that improves deployment efficiency, scalability and system monitoring.



Fig1: Frontend Dashboard Interface

3.2 Key Features

The proposed **AQI-Pulse** system provides multiple features that combine air quality monitoring with DevOps and cloud-native deployment technologies. The major features of the system are as follows:

- **Interactive AQI Dashboard:**
Provides a user-friendly interface for viewing air quality information and application outputs.
- **Docker Containerization:**
Packages the application and dependencies into containers to ensure portability and consistency across environments.
- **CI/CD Pipeline using Jenkins:**
Automates code integration, image creation and deployment processes.
- **Cloud Deployment using AWS EC2:**
Hosts the application on cloud infrastructure for remote accessibility and better resource management.
- **Kubernetes Orchestration:**
Manages deployments, pods and services while providing self-healing capabilities.
- **Node-Port Service Configuration:**
Exposes application services externally for browser access.
- **Prometheus Monitoring:**
Collects application and infrastructure metrics periodically.
- **Grafana Dashboard Visualization:**
Displays collected metrics through graphical dashboards.

These features collectively improve scalability, automation, portability and system observability.

3.3 Functional Requirements

Functional requirements describe the major operations and services that the proposed system must perform to achieve its objectives. These requirements

define the expected behavior of the system from both user and infrastructure perspectives.

The functional requirements of AQI-Pulse are as follows:

- The system should provide an interactive dashboard interface for users.
- The application should display AQI-related information and outputs.
- The system should support containerized deployment using Docker.
- The Jenkins pipeline should automatically build and deploy application updates.
- The system should store Docker images in Docker Hub.
- Kubernetes should create and manage application pods automatically.
- The application should be accessible through Node-Port services.
- Prometheus should collect application metrics periodically.
- Grafana should visualize collected metrics through dashboards.
- The system should support automatic pod recreation in case of failures.

These requirements ensure proper functionality of both the application and deployment infrastructure.

3.3.1 User Requirements

User requirements describe the functionalities expected by end users while interacting with the AQI-Pulse system. The application should provide a simple and accessible interface for viewing information and interacting with system services.

The user requirements are as follows:

- Users should be able to access the application through a web browser.
- The system should provide an interactive and user-friendly dashboard interface.
- Users should be able to view AQI-related information and results.

- The application interface should be responsive and easy to navigate.
- Users should be able to access the system remotely through cloud deployment.
- The system should provide reliable and uninterrupted service availability.

These requirements ensure better usability and improve overall user experience.

3.3.2 Infrastructure Requirements

Infrastructure requirements define the hardware, software and cloud resources required for successful deployment and execution of the AQI-Pulse system. These requirements ensure smooth application performance and proper functioning of deployment and monitoring components.

The infrastructure requirements for the proposed system are:

- **AWS EC2 Instance:** Used for hosting the application and Kubernetes cluster.
- **Kubernetes K3s Cluster:** Used for managing deployments, services and pods.
- **Docker Environment:** Required for container creation and execution.
- **Jenkins Server:** Required for CI/CD pipeline automation.
- **Docker Hub Repository:** Used for storing and managing container images.
- **Prometheus Server:** Required for collecting application metrics.
- **Grafana Dashboard:** Used for visualizing monitoring data.
- **Internet Connectivity:** Required for cloud access and communication between services.

These infrastructure components collectively support deployment automation, monitoring and cloud-native execution of the application.

3.4 Non Functional Requirements

Non-functional requirements define the quality attributes and performance characteristics of the proposed system. These requirements focus on how the system performs rather than what functions it performs.

The non-functional requirements of **AQI-Pulse** are as follows:

- **Scalability:** The system should support future expansion and increased workload handling.
- **Reliability:** The application should provide stable performance with minimal downtime.
- **Availability:** Services should remain accessible to users whenever required.
- **Portability:** The application should run consistently across different environments using Docker containers.
- **Performance:** The system should provide efficient response time and resource utilization.
- **Maintainability:** The deployment and monitoring infrastructure should be easy to manage and update.
- **Security:** Access to cloud infrastructure and deployment services should be controlled properly.
- **Observability:** The system should support monitoring through Prometheus and Grafana dashboards.

These requirements improve overall system quality and ensure efficient operation of the proposed platform.

3.5 High-Level Workflow

The AQI-Pulse system follows a cloud-native workflow that integrates application development, CI/CD automation, containerization, orchestration and monitoring. The workflow begins when the developer pushes updated source code to the GitHub repository.

A GitHub webhook triggers the Jenkins pipeline, which automatically starts the deployment process. Jenkins clones the source code, builds the Docker image and pushes the image to Docker Hub. After image creation, Kubernetes deployment files are applied to update the application running inside the cluster.

The application is deployed on an AWS EC2 instance configured with a single-node Kubernetes K3s cluster. Kubernetes creates and manages application pods and exposes services using Node-Port configuration.

For monitoring purposes, Prometheus periodically collects application metrics and stores monitoring data. Grafana retrieves the collected metrics from Prometheus and displays them through interactive dashboards.

The overall workflow can be represented as:

Developer → GitHub → Jenkins → Docker → Docker Hub → AWS EC2 → Kubernetes → Prometheus → Grafana → Users

Chapter 4: System Architecture

4.1 Overall System Architecture

The overall architecture of AQI-Pulse follows a cloud-native and DevOps-based approach that integrates application development, deployment automation, orchestration and monitoring into a single workflow. The architecture is designed to ensure portability, scalability and simplified management of application services.

The system begins with the developer creating or updating source code and pushing changes to the GitHub repository. GitHub acts as a centralized version control platform for maintaining project files and deployment configurations.

Once code changes are pushed, a GitHub webhook triggers the Jenkins CI/CD pipeline. Jenkins automatically retrieves the latest source code, builds the Docker image and pushes it to Docker Hub.

The application image is deployed on an AWS EC2 instance running a single-node Kubernetes K3s cluster. Kubernetes manages Deployments, ReplicaSets and Pods while exposing application access through Node-Port services.

The monitoring architecture includes Prometheus for collecting metrics and Grafana for dashboard visualization. Prometheus periodically scrapes metrics and Grafana displays performance information through graphical panels.

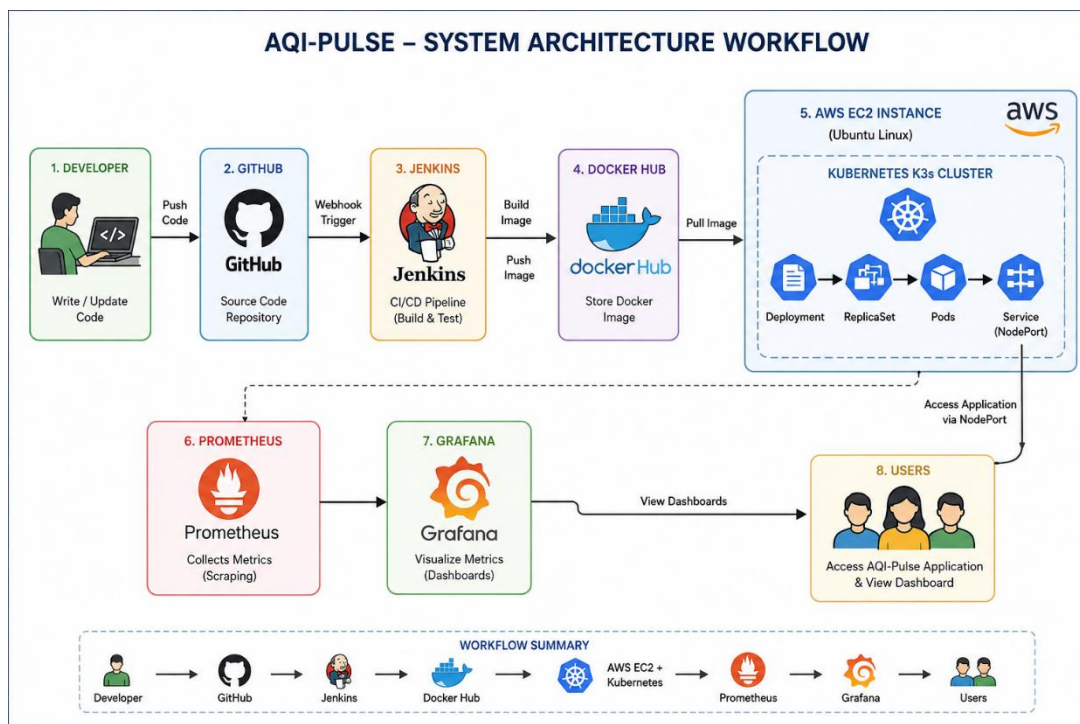


Fig 2: Overall System Architecture

This architecture provides automation, monitoring and efficient deployment management while reducing manual effort.

4.2 Frontend Architecture

The frontend of AQI-Pulse is developed using Streamlit, a Python-based framework used for creating interactive web applications. Streamlit provides a lightweight and user-friendly interface that allows users to access AQI information and interact with the application through a browser.

The frontend layer is responsible for displaying application content, accepting user interactions and presenting outputs in a visual format. It acts as the communication interface between users and backend functionalities.

The major responsibilities of the frontend layer include:

- Displaying AQI-related information
- Providing an interactive user interface
- Accepting user requests
- Showing prediction outputs
- Presenting graphical visualizations
- Connecting user actions with backend logic

The application interface runs inside a Docker container and is exposed externally using a Kubernetes Node-Port service, enabling users to access the dashboard through browser requests.

4.3 Backend Architecture

The backend architecture of AQI-Pulse is responsible for processing application logic, handling AQI-related operations and executing prediction functionalities. The backend is implemented using Python, which acts as the core programming language for managing system operations.

The backend layer performs computation and integrates different modules required by the application. It processes requests received from the Streamlit interface and generates outputs that are displayed to users.

The major responsibilities of the backend layer include:

- Processing application requests
- Managing AQI prediction logic
- Executing Python functions and scripts
- Loading required datasets and models
- Connecting frontend interactions with backend processing
- Generating outputs for dashboard display

The backend components are packaged inside the Docker container along with all dependencies. This ensures consistency across deployment environments and avoids dependency-related issues.

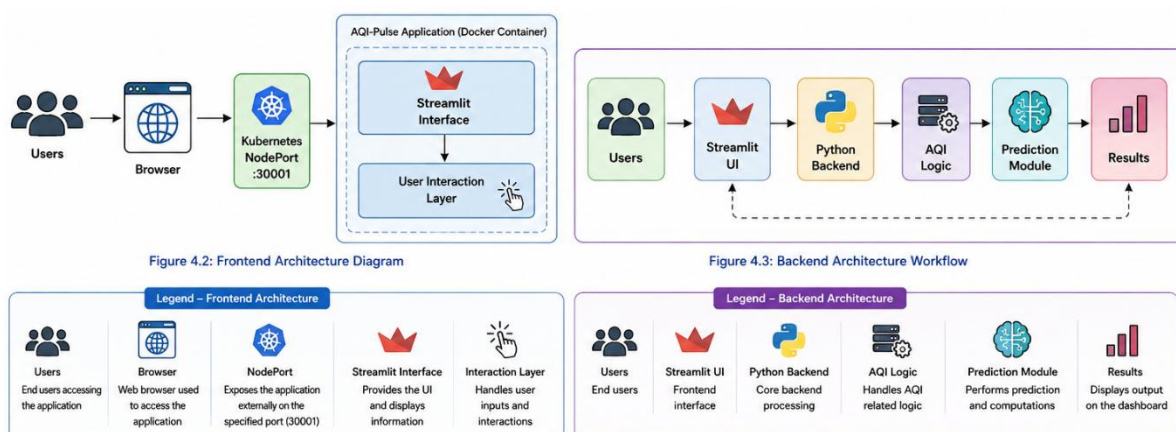


Fig 3: Frontend and Backend Architecture

4.4 Deployment Architecture

The deployment architecture of AQI-Pulse is designed using a cloud-native approach where the application is deployed on an AWS EC2 instance running a single-node Kubernetes K3s cluster. The architecture integrates Docker, Kubernetes and CI/CD automation to ensure efficient deployment and simplified management.

The deployment process begins when the developer pushes updated source code to the GitHub repository. A webhook triggers the Jenkins pipeline, which

automatically clones the code and builds a Docker image. The generated image is pushed to Docker Hub, which acts as a centralized image repository.

The Kubernetes cluster running on AWS EC2 pulls the Docker image from Docker Hub and creates required resources using YAML configuration files. Kubernetes Deployments create ReplicaSets, which further create and manage application Pods. Services expose application access through Node-Port configuration.

The deployment architecture provides:

- Automated deployment workflow
- Container orchestration
- Pod lifecycle management
- Service exposure through Node-Port
- Cloud-based accessibility
- Self-healing capability

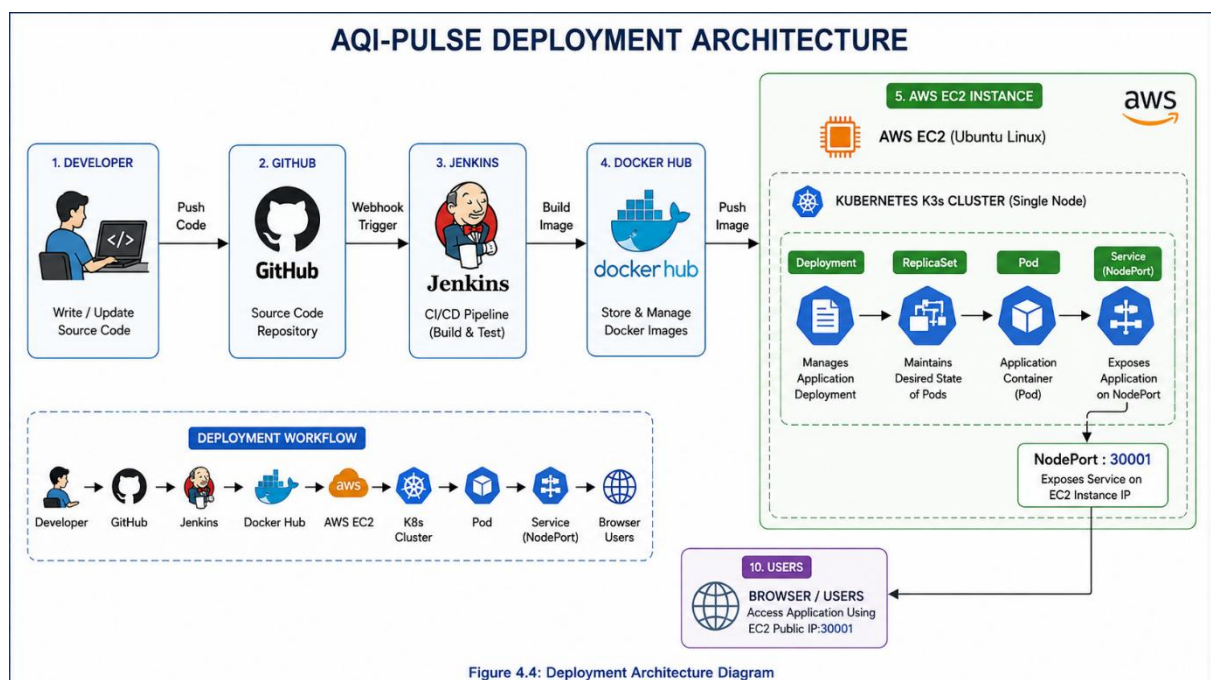


Fig 4: Deployment Architecture

4.5 Monitoring Architecture

The monitoring architecture of AQI-Pulse is implemented using Prometheus and Grafana to provide observability and performance tracking of the deployed application. Monitoring helps in analyzing application behavior, tracking resource usage and identifying failures within the system.

Prometheus continuously collects metrics by scraping application endpoints at fixed time intervals. In the AQI-Pulse system, application metrics are exposed through the metrics endpoint and Prometheus periodically retrieves information related to application activity.

The collected metrics are stored and later accessed by Grafana, which acts as a visualization platform. Grafana queries Prometheus and displays data through dashboards containing graphs, charts and monitoring panels.

The monitoring workflow includes:

- Metrics generation from application
- Prometheus scraping process
- Metrics storage
- Dashboard visualization
- Monitoring and analysis

The monitoring architecture improves observability and enables administrators to monitor application behavior in real time.

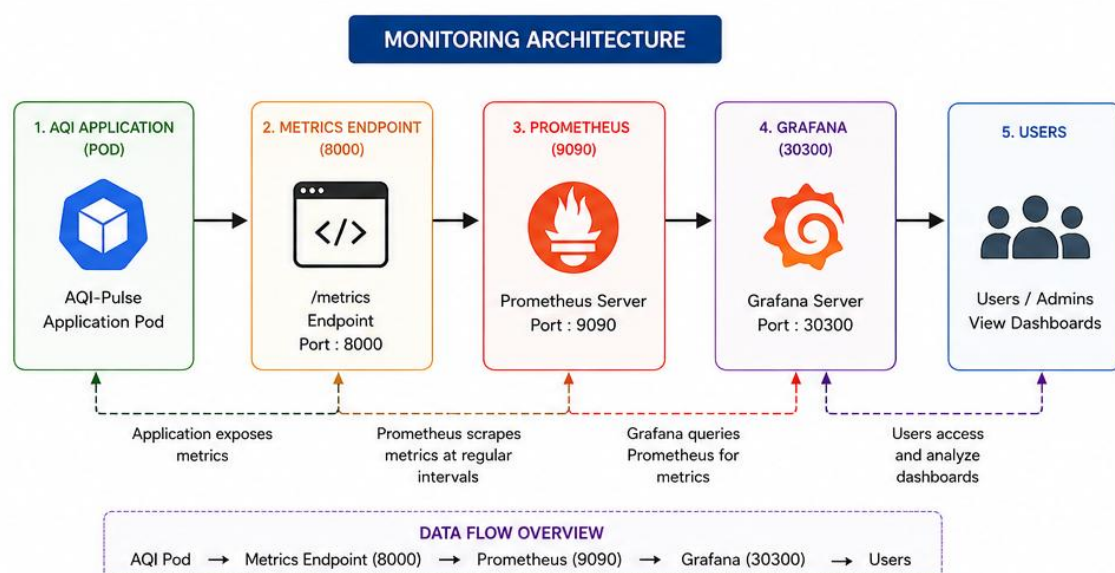


Figure 4.5: Monitoring Architecture Diagram

Fig 5: Monitoring Architecture

Chapter 5: Technology Stack

5.1 Technology Overview and Justification

The development of AQI-Pulse requires multiple technologies to support application development, deployment automation, orchestration and monitoring. Each technology used in the project was selected based on its functionality and contribution toward building a cloud-native architecture.

Python is used as the primary programming language because of its simplicity and support for data processing and machine learning operations.

Streamlit is selected for creating an interactive web interface with minimal frontend complexity.

Docker is used for containerization to package the application and its dependencies into a portable environment.

Jenkins is implemented for Continuous Integration and Continuous Deployment (CI/CD) automation. It reduces manual deployment effort and improves development workflow.

Docker Hub acts as a centralized image repository for storing application containers.

AWS EC2 provides cloud infrastructure for deployment and remote accessibility.

Kubernetes K3s is used for orchestration and management of application resources including Deployments, Pods and Services.

Prometheus provides metrics collection functionality while **Grafana** visualizes monitoring information using dashboards.

The integration of these technologies creates an automated workflow capable of improving scalability, deployment efficiency and observability.

5.2 Python and Streamlit

The AQI-Pulse application is developed using Python and Streamlit. Python serves as the primary programming language and manages application logic,

processing operations and prediction functionalities. It was selected because of its simplicity, readability and extensive library support.

Streamlit is used to create the frontend interface of the application. It allows development of interactive web applications directly using Python without requiring complex frontend technologies.

In the AQI-Pulse system, Streamlit is responsible for presenting dashboard components and displaying outputs generated by backend processing modules.

The advantages of using Python and Streamlit include:

- Easy application development
- Interactive user interface creation
- Fast deployment capability
- Large ecosystem and library support
- Reduced frontend complexity
- Integration with machine learning workflows

The combination of Python and Streamlit simplifies application development while providing a clean and interactive user experience.

5.3 Docker

Docker is a containerization platform used to package applications and their dependencies into isolated environments called containers. It ensures that the application runs consistently across different systems regardless of operating system differences or dependency configurations.

In the AQI-Pulse project, Docker is used to containerize the Streamlit application along with required libraries and project files. The application is packaged into a Docker image and stored in Docker Hub, allowing Kubernetes to pull and deploy it on the cloud environment.

Docker simplifies deployment and eliminates issues related to environment inconsistencies. Instead of manually installing dependencies on servers, the entire application environment is bundled into a portable container.

The major benefits of Docker include:

- Portability across environments
- Dependency isolation
- Faster deployment process
- Lightweight execution
- Simplified application packaging
- Better resource utilization

The Docker workflow used in the project can be represented as:

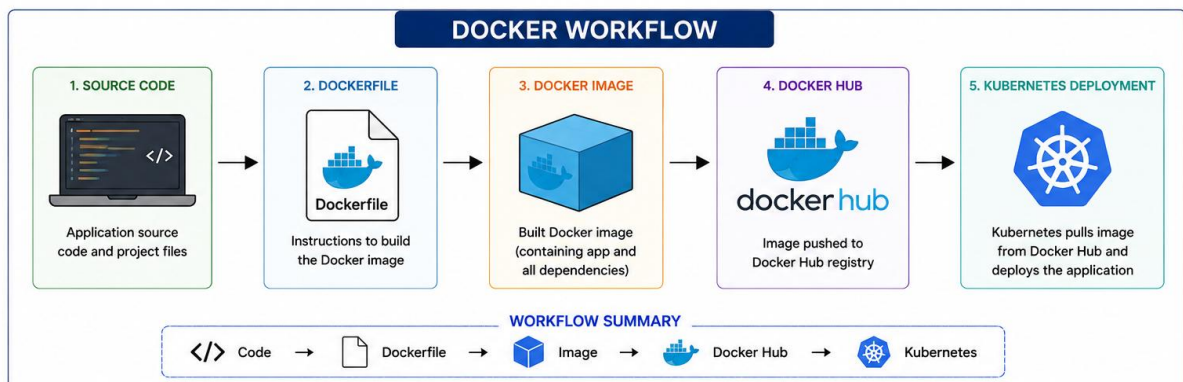


Fig 6: Docker Workflow

5.4 Kubernetes (K3s)

Kubernetes is an open-source container orchestration platform used for managing, deploying and scaling containerized applications. It automates deployment operations and ensures efficient management of application resources.

In the AQI-Pulse project, K3s is used instead of standard Kubernetes. K3s is a lightweight version of Kubernetes designed for environments with limited resources while providing the same core functionality.

The project uses a single-node K3s cluster deployed on an AWS EC2 instance. Kubernetes manages application resources through components such as Deployments, ReplicaSets, Pods and Services.

The deployment process begins when Kubernetes reads YAML configuration files and creates required resources automatically. If a pod fails, Kubernetes recreates it using self-healing capability.

The major Kubernetes components used are:

- **Deployment:** Manages application deployment
- **ReplicaSet:** Maintains desired number of pod replicas
- **Pod:** Runs application containers
- **Service:** Exposes application access
- **Node-Port:** Makes application accessible externally

Kubernetes improves deployment automation, fault tolerance and application management.

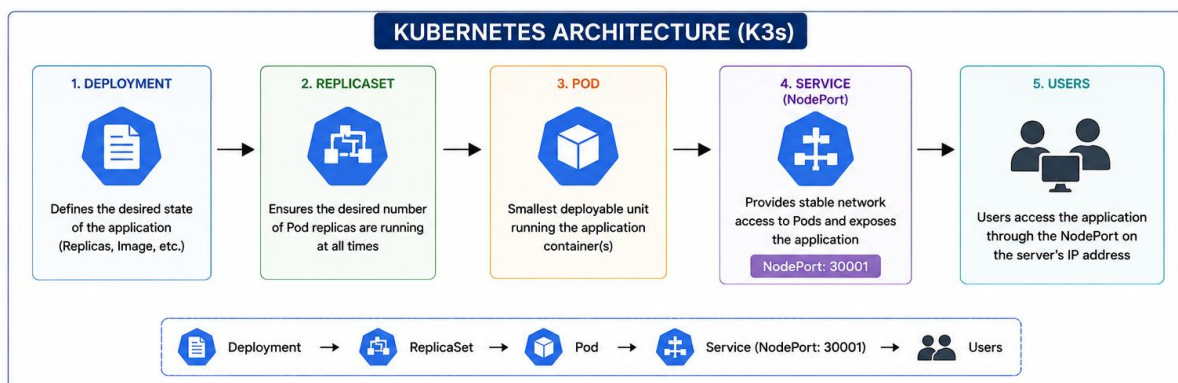


Fig 7: Kubernetes Architecture

5.5 Jenkins

Jenkins is an open-source automation server used for implementing Continuous Integration (CI) and Continuous Deployment (CD) processes. It automates repetitive development tasks such as building applications, creating Docker images and deploying updated versions of software.

In the AQI-Pulse project, Jenkins is integrated with GitHub through webhooks. Whenever source code changes are pushed to the repository, Jenkins automatically starts the pipeline process.

The Jenkins pipeline used in the project contains multiple stages:

- **Clone Stage:** Retrieves source code from GitHub
- **Build Docker Image:** Creates Docker image from application files
- **Docker Login:** Authenticates Docker Hub account
- **Push Image:** Uploads image to Docker Hub
- **Deploy to Kubernetes:** Updates application deployment in cluster

The pipeline eliminates manual deployment activities and creates an automated workflow from development to deployment.

The Jenkins workflow used in the project is:

GitHub → Webhook → Jenkins → Build Image → Docker Hub → Kubernetes Deployment

Benefits of Jenkins:

- Automated deployment
- Reduced manual effort
- Faster development workflow
- Improved consistency
- Continuous integration support

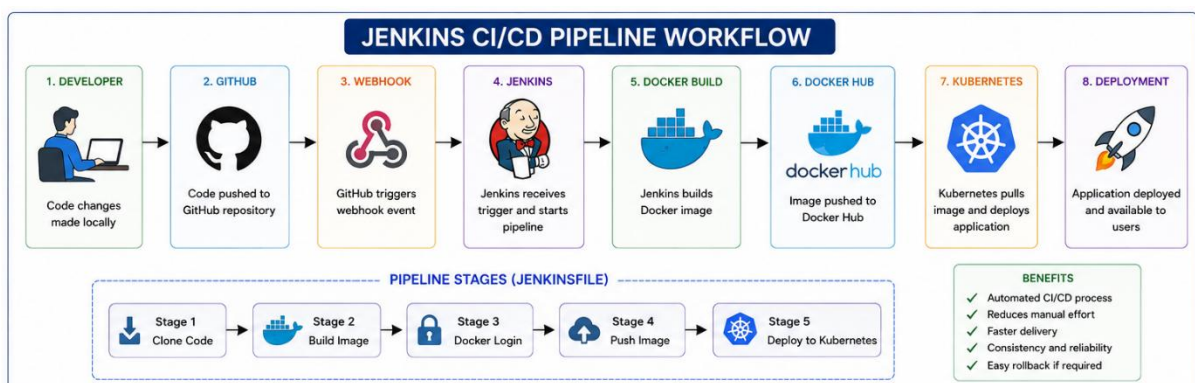


Fig 8: Jenkins CI/CD Pipeline Workflow

5.6 Prometheus and Grafana

Prometheus and Grafana are used in AQI-Pulse to implement monitoring and observability capabilities. These tools help track application behavior, collect metrics and visualize system performance.

Prometheus is an open-source monitoring tool that periodically collects metrics from configured targets through a process known as scraping. In the AQI-Pulse project, Prometheus scrapes application metrics exposed by the AQI application endpoint.

The metrics collected by Prometheus include:

- Request count
- Application status
- CPU usage
- Memory information
- System activity metrics

Grafana acts as a visualization platform that connects with Prometheus and converts collected metrics into interactive dashboards. Users can analyze graphs, charts and monitoring panels through a browser interface.

The monitoring workflow in the project is:

AQI Application → Metrics Endpoint → Prometheus → Grafana → Users

Benefits of Prometheus and Grafana:

- Real-time monitoring
- Metrics collection and storage
- Dashboard visualization
- Performance analysis
- Improved observability

5.7 AWS EC2

Amazon Elastic Compute Cloud (AWS EC2) is a cloud computing service provided by Amazon Web Services that allows users to create and manage virtual servers in the cloud. It provides scalable infrastructure for hosting applications and deployment environments.

In the AQI-Pulse project, an AWS EC2 Ubuntu instance is used as the main deployment server. The EC2 instance hosts:

- Kubernetes K3s cluster
- AQI-Pulse application pods
- Jenkins automation environment
- Prometheus monitoring service
- Grafana dashboards

The EC2 instance acts as the central cloud infrastructure where all deployment and monitoring operations are executed.

The application becomes accessible to users through the public IP address of the EC2 instance combined with Kubernetes Node-Port services.

Benefits of using AWS EC2:

- Cloud-based accessibility
- Scalable infrastructure
- Remote deployment support
- Flexible resource management
- Continuous application availability

The deployment access flow is:

Users → EC2 Public IP → Node-Port Service → Kubernetes Pod → AQI Application

Chapter 6: Containerization and Deployment

6.1 What is Containerization

Containerization is a software deployment approach in which an application and all its dependencies are packaged together into a lightweight unit called a container. The container includes source code, libraries, runtime environment and required configurations needed for application execution.

Containerization ensures that an application behaves consistently across different environments without dependency conflicts. Instead of installing software packages separately on multiple systems, the complete environment is bundled inside a container.

In the AQI-Pulse project, Docker is used for containerization. The Streamlit application and all required project files are packaged into a Docker image and later deployed through Kubernetes.

The use of containerization provides several benefits:

- Environment consistency
- Portability across systems
- Faster deployment process
- Isolation of dependencies
- Simplified application management
- Reduced configuration issues

Containerization helps improve deployment efficiency and supports cloud-native application development.

6.2 Docker Architecture

Docker architecture follows a client–server model where different components work together to build, manage and run containers. The architecture consists of the Docker Client, Docker Host, Docker Engine, Images and Containers.

In the AQI-Pulse project, Docker architecture is used to create a containerized environment for the application. The developer interacts with Docker commands through the client interface. Docker processes these commands and creates Docker images from the Dockerfile.

The Docker image is then stored in Docker Hub, which acts as a container registry. Kubernetes later pulls the image and deploys it within the cluster.

Major Docker architecture components include:

- **Docker Client:** Accepts user commands
- **Docker Daemon/Engine:** Processes Docker operations
- **Docker Image:** Template used to create containers
- **Docker Container:** Running instance of application
- **Docker Hub:** Stores container images

Docker architecture workflow:

Developer → Docker Client → Docker Engine → Docker Image → Docker Container

A screenshot of a code editor showing a Dockerfile. The file is titled 'Dockerfile' with a small icon. The content of the Dockerfile is as follows:

```
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 EXPOSE 8501
12
13 CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

The Docker architecture simplifies container creation and deployment while maintaining application consistency.

Important Docker Commands Used

Build Docker image:

```
docker build -t archi2402/aqi-pulse:latest .
```

View available Docker images:

```
docker images
```

Run Docker container:

```
docker run -p 8501:8501 archi2402/aqi-pulse:latest
```

Tag Docker image:

```
docker tag archi2402/aqi-pulse:latest archi2402/aqi-pulse:v1
```

Push image to Docker Hub:

```
docker push archi2402/aqi-pulse:latest
```

Pull image from Docker Hub:

```
docker pull archi2402/aqi-pulse:latest
```

These commands were used throughout the AQI-Pulse workflow for building, managing and deploying application containers.

6.3 Port Configuration

Port configuration is an important part of containerized deployment because it enables communication between applications and external users. Ports are used to expose services running inside containers and Kubernetes pods.

In the AQI-Pulse project, different ports are configured for application access and monitoring services. These ports allow communication between users, Kubernetes services and monitoring components.

The ports used in the project are:

Component	Port Number	Purpose
AQI Streamlit Application	8501	Runs application inside container
Kubernetes Node-Port	30001	Exposes application externally
Prometheus	9090	Monitoring server access
Metrics Endpoint	8000	Exposes application metrics
Grafana	30300	Dashboard visualization access
Jenkins	8080	CI/CD server access

The application first runs on the internal container port and is later exposed externally through Kubernetes Node-Port service. This allows users to access the application through:

EC2 Public IP : Node-Port

Example:

`http://EC2-Public-IP:30001`

Proper port configuration ensures successful communication between application

6.4 Kubernetes Deployment

Kubernetes deployment is responsible for managing and running containerized applications inside the cluster environment. In the AQI-Pulse project, Kubernetes deployment is implemented using YAML configuration files that define application resources and deployment behavior.

The deployment configuration specifies details such as container image, replicas, ports and labels. Kubernetes reads these YAML files and automatically creates required resources.

The project uses:

- **Deployment** for application management

- **ReplicaSet** for maintaining pod count
- **Pod** for running application containers
- **Service** for exposing application access

The AQI application deployment is created using:

```
kubectrl apply -f aqi-deployment.yaml
```

To create the service:

```
kubectrl apply -f aqi-service.yaml
```

The deployment process enables Kubernetes to automatically manage application lifecycle and maintain availability.

6.5 Node-Port Configuration

Node-Port is a Kubernetes service type used to expose applications externally. It opens a port on the Kubernetes node and forwards traffic to the application pod.

In AQI-Pulse, the application is exposed through:

- Container Port: **8501**
- Target Port: **8501**
- Node-Port: **30001**

User request flow:

Browser → EC2 Public IP → Node-Port:30001 → Service → Pod → AQI Application

Useful commands:

View Node-Port details:

```
kubectrl describe service aqi-pulse-service
```

Application access:

```
http://<EC2-Public-IP>:30001
```

Chapter 7: CI/CD Pipeline Using Jenkins

7.1 What is CI/CD

Continuous Integration (CI) and Continuous Deployment (CD) are software development practices used to automate the process of building, testing and deploying applications. CI/CD reduces manual effort and enables faster and more reliable software delivery.

Continuous Integration (CI) refers to the process where developers regularly push source code changes to a shared repository. Whenever code changes are committed, automated processes are triggered to verify and integrate the updates.

Continuous Deployment (CD) extends this process by automatically deploying updated applications to the target environment after successful build stages.

In the AQI-Pulse project, CI/CD is implemented using Jenkins Pipeline. When source code is pushed to GitHub, Jenkins automatically performs multiple operations such as:

- Cloning source code
- Building Docker image
- Authenticating Docker Hub
- Pushing image to Docker Hub
- Deploying application to Kubernetes

Benefits of CI/CD:

- Faster deployment cycle
- Reduced manual work
- Better software reliability
- Automated delivery process
- Continuous updates and monitoring

7.2 Jenkins Pipeline Architecture

Jenkins Pipeline Architecture represents the automated workflow followed by Jenkins for building and deploying the AQI-Pulse application. The pipeline consists of multiple stages that execute sequentially whenever new code changes are pushed to the GitHub repository.

In the AQI-Pulse project, Jenkins is connected with GitHub through webhooks. When a developer pushes source code changes, GitHub triggers Jenkins automatically and starts the CI/CD process.

The Jenkins architecture used in the project consists of:

- **Developer:** Updates application code
- **GitHub Repository:** Stores source code
- **Webhook:** Triggers Jenkins automatically
- **Jenkins Pipeline:** Executes deployment stages
- **Docker Hub:** Stores generated Docker images
- **Kubernetes Cluster:** Deploys application containers

The Jenkins pipeline removes manual deployment activities and creates a fully automated workflow.

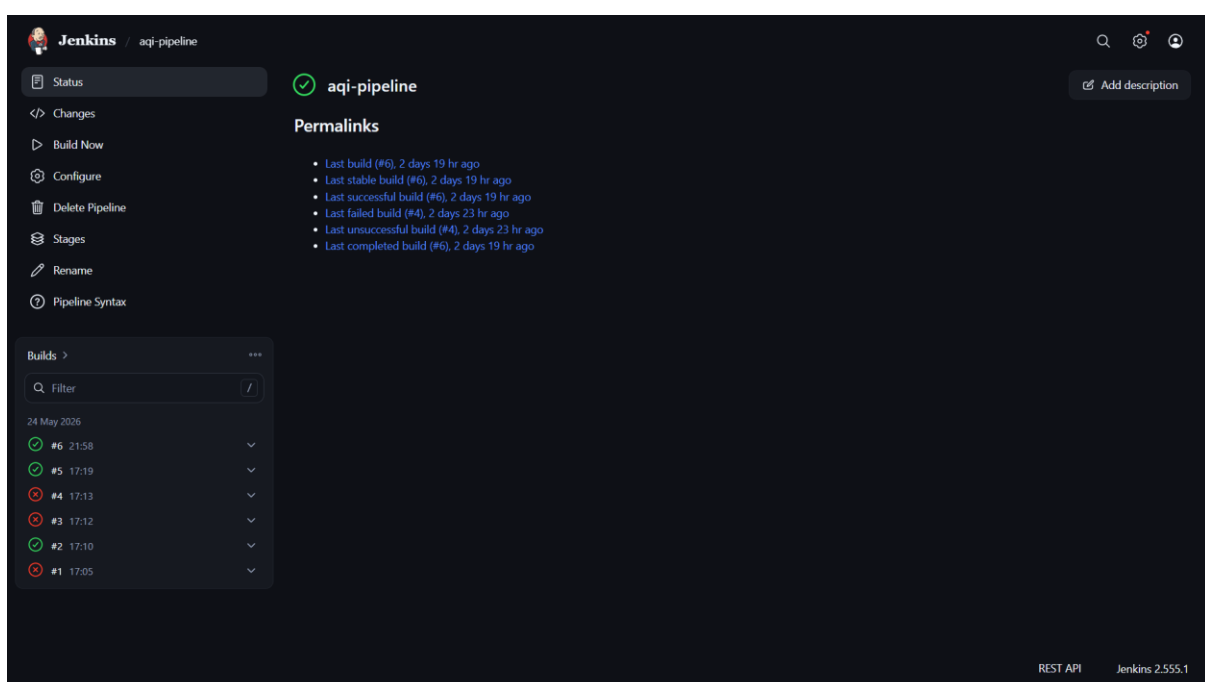


Fig 9: Jenkins Successful Build

7.3 Pipeline Stages

The Jenkins pipeline in AQI-Pulse is divided into multiple stages. Each stage performs a specific operation required for automated deployment. The stages are executed sequentially, and successful completion of one stage triggers execution of the next stage.

The pipeline stages implemented in the project are:

Stage 1: Clone Repository

This stage retrieves the latest source code from the GitHub repository.

```
stages {  
    stage('Clone') {  
        steps {  
            git branch: 'main',  
              url: 'https://github.com/ARCHI-DUBEY/aqi.git'  
        }  
    }  
}
```

Stage 2: Build Docker Image

This stage creates a Docker image using the Dockerfile.

```
stage('Build Docker Image') {  
    steps {  
        bat 'docker build -t %DOCKER_IMAGE%:%IMAGE_TAG% .'  
    }  
}
```

Stage 3: Docker Login

This stage authenticates Docker Hub credentials.

```
stage('Docker Login') {  
    steps {  
        withCredentials([  
            usernamePassword(  
                credentialsId: 'dockerhub-creds',  
                usernameVariable: 'DOCKER_USER',  
                passwordVariable: 'DOCKER_PASS'  
            )  
        ]) {  
            bat 'docker login -u %DOCKER_USER% -p %DOCKER_PASS%'  
        }  
    }  
}
```

Stage 4: Push Docker Image

This stage uploads the generated image to Docker Hub.

```
stage('Push Image') {  
    steps {  
        bat 'docker push %DOCKER_IMAGE%:%IMAGE_TAG%'  
        bat 'docker tag %DOCKER_IMAGE%:%IMAGE_TAG% %DOCKER_IMAGE%:latest'  
        bat 'docker push %DOCKER_IMAGE%:latest'  
    }  
}
```

Stage 5: Deploy to Kubernetes

This stage deploys the updated image into Kubernetes.

```
stage('Deploy Monitoring') {  
    steps {  
        bat 'kubectl apply -f k8s\\prometheus.yaml'  
        bat 'kubectl apply -f k8s\\grafana.yaml'  
    }  
}  
  
stage('Deploy AQI App') {  
    steps {  
        bat 'kubectl apply -f k8s\\aqi-deployment.yaml'  
        bat 'kubectl apply -f k8s\\aqi-service.yaml'  
        bat 'kubectl set image deployment/aqi-pulse aqi-pulse=%DOCKER_IMAGE%:%IMAGE_TAG%'  
        bat 'kubectl rollout status deployment/aqi-pulse'  
    }  
}
```

This staged approach simplifies deployment management and improves automation efficiency.

Chapter 8: Kubernetes Orchestration

8.1 What is Kubernetes

Kubernetes is an open-source container orchestration platform used for deploying, managing and monitoring containerized applications. It automates tasks such as application deployment, scaling, networking and resource management.

As applications grow larger, manually managing containers becomes difficult. Kubernetes solves this problem by organizing containers into manageable units and automatically maintaining the desired system state.

In the AQI-Pulse project, Kubernetes is used to manage deployment of the Dockerized application inside a single-node K3s cluster running on an AWS EC2 instance.

The Kubernetes cluster manages:

- Application deployment
- Pod creation and management
- Replica maintenance
- Service exposure
- Self-healing functionality
- Resource orchestration

Important Kubernetes resources used in AQI-Pulse:

- **Deployment** → Manages application deployment
- **ReplicaSet** → Maintains desired pod count
- **Pod** → Runs application container
- **Service** → Exposes applications
- **Node-Port** → Enables external access

Kubernetes helps automate deployment management and ensures high application availability.

8.2 Why Kubernetes (K3s)

K3s is a lightweight distribution of Kubernetes designed for environments with limited resources while still providing core Kubernetes functionalities. It is easier

to install and consumes fewer system resources compared to standard Kubernetes deployments.

In the AQI-Pulse project, K3s was selected because the application is deployed on a single AWS EC2 instance, where resource efficiency is important. K3s provides all major Kubernetes features while reducing infrastructure complexity.

Reasons for selecting K3s:

- Lightweight architecture
- Lower CPU and memory usage
- Faster installation process
- Suitable for single-node deployment
- Supports standard Kubernetes APIs
- Easier maintenance and configuration

The project uses a single-node K3s cluster with:

- **Nodes: 1**
- **AQI Application Pod: 1**
- **Prometheus Pod: 1**
- **Grafana Pod: 1**
- **Replicas: 1**

Useful commands:

Check K3s node:

```
kubectl get nodes
```

View Kubernetes version:

```
kubectl version
```

K3s simplifies Kubernetes deployment while maintaining orchestration capabilities.

8.3 Deployment Configuration

Deployment configuration defines how Kubernetes creates and manages application resources. In AQI-Pulse, deployment is implemented using YAML

configuration files, which specify details such as application image, replicas, labels and container ports.

The deployment file used in the project is `aqi-deployment.yaml`, where Kubernetes reads configuration settings and automatically creates required resources.

The deployment configuration contains:

- Application name: **aqi-pulse**
- Replicas: **1**
- Container image: **archi2402/aqi-pulse:latest**
- Container port: **8501**
- Labels: **app: aqi-pulse**

When Kubernetes executes the deployment file, the following workflow occurs:

Deployment → ReplicaSet → Pod Creation → Running Application

```
k8s > ! aqi-deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3
4  metadata:
5    name: aqi-pulse
6
7  spec:
8    replicas: 1
9
10   selector:
11     matchLabels:
12       app: aqi-pulse
13
14   template:
15     metadata:
16       labels:
17         app: aqi-pulse
18
19     spec:
20       containers:
21         - name: aqi-pulse
22
23           image: archi2402/aqi-pulse:latest
24
25           ports:
26             - containerPort: 8501
```

Useful Kubernetes commands used during deployment:

Create deployment:

kubectl apply -f aqi-deployment.yaml

View deployments:

```
kubectrl get deployments
```

Check rollout status:

```
kubectrl rollout status deployment/aqi-pulse
```

Check running pods:

```
kubectrl get pods
```

Check services:

```
kubectrl get svc
```

These commands help create, monitor and manage deployment resources within the Kubernetes cluster.

8.4 Node-Port Service

A Node-Port Service is a Kubernetes service type used to expose applications outside the Kubernetes cluster. It opens a specific port on the node and forwards incoming traffic to the application pod.

In the AQI-Pulse project, Node-Port is used to make the Streamlit application accessible through the browser. The application internally runs on container port 8501, while Kubernetes exposes it externally through Node-Port 30001.

Configuration used in the project:

- Container Port: **8501**
- Target Port: **8501**
- Node-Port: **30001**

Request flow:

Users → EC2 Public IP → Node-Port:30001 → Service → AQI Pod → Application

Example access URL:

http://<EC2-Public-IP>:30001

```
k8s > ! aqi-service.yaml
 1  apiVersion: v1
 2  kind: Service
 3
 4  metadata:
 5    name: aqi-pulse-service
 6
 7  spec:
 8    type: NodePort
 9
10    selector:
11      app: aqi-pulse
12
13    ports:
14      - protocol: TCP
15        port: 8501
16        targetPort: 8501
17        nodePort: 30001
```

Useful commands:

Check services:

```
kubectl get svc
```

View service details:

```
kubectl describe service aqi-pulse-service
```

View service with ports:

```
kubectl get svc -o wide
```

Get EC2 public IP:

```
curl ifconfig.me
```

The Node-Port service allows external users to access the AQI dashboard without directly interacting with pods.

Chapter 9: Monitoring and Observability

9.1 Observability Principles

Observability refers to the ability to monitor, analyze and understand the internal behavior of a system based on outputs such as metrics, logs and performance data. It helps administrators identify failures, analyze application health and understand system behavior.

In modern cloud-native environments, observability plays an important role because applications are distributed across multiple services and infrastructure components. Monitoring allows developers to identify issues quickly and maintain system reliability.

The AQI-Pulse project implements observability using Prometheus and Grafana. These tools help monitor application metrics and visualize system behavior through dashboards.

The observability process in the project focuses on:

- Application health monitoring
- Metrics collection
- Performance analysis
- Resource utilization tracking
- Failure detection
- Dashboard visualization

Implementing observability improves system visibility and supports efficient management of cloud-native applications.

9.2 Prometheus Metrics Collection

Prometheus is an open-source monitoring tool used for collecting and storing metrics from applications and infrastructure components. It works using a pull-based mechanism where metrics are collected periodically from configured targets through a process called scraping.

In the AQI-Pulse project, Prometheus is configured using a `prometheus.yml` file. It continuously accesses the metrics endpoint of the application and collects information at fixed intervals.

The scraping process used in the project follows:

AQI Application → Metrics Endpoint → Prometheus Server → Metrics Storage

The Prometheus configuration includes:

- Scrape interval: **15 seconds**
- Monitoring target: **AQI application metrics endpoint**
- Metrics storage for analysis and visualization

```
k8s > ! prometheus.yaml
1  apiVersion: v1
2  kind: ConfigMap
3
4  metadata:
5    name: prometheus-config
6
7  data:
8    prometheus.yml: |
9      global:
10       scrape_interval: 15s
11
12       scrape_configs:
13
14         - job_name: 'prometheus'
15           static_configs:
16             - targets: ['localhost:9090']
17
18         - job_name: 'aqi-app'
19           static_configs:
20             - targets:
21               - 'aqi-pulse-service:8000'
```

Useful commands:

Start Prometheus:

```
kubectl apply -f prometheus.yaml
```

Check Prometheus pod:

```
kubectl get pods
```

Check Prometheus service:

kubectl get svc

Open Prometheus:

http://<EC2-Public-IP>:30090

Prometheus continuously collects metrics and makes them available for visualization through Grafana dashboards.

9.3 Grafana Dashboard

Grafana is an open-source visualization platform used for displaying monitoring data through interactive dashboards. It connects with different data sources and converts collected metrics into meaningful graphical representations.

In the AQI-Pulse project, Grafana is integrated with Prometheus as a data source. Prometheus collects metrics from the application, and Grafana retrieves these metrics to generate visual dashboards.

The Grafana dashboard helps monitor:

- Application health status
- Request information
- CPU utilization
- Memory usage
- Performance metrics
- System activity

The dashboard provides graphical panels such as:

- Line charts
- Bar graphs
- Statistical panels
- Time-series visualizations

```

k8s > ! grafana.yaml
 1  apiVersion: apps/v1
 2  kind: Deployment
 3  metadata:
 4    name: grafana
 5  spec:
 6    replicas: 1
 7    selector:
 8      matchLabels:
 9        app: grafana
10    template:
11      metadata:
12        labels:
13          app: grafana
14      spec:
15        containers:
16          - name: grafana
17            image: grafana/grafana
18            ports:
19              - containerPort: 3000
20
21  ---
22  apiVersion: v1
23  kind: Service
24  metadata:
25    name: grafana-service
26  spec:
27    type: NodePort
28    selector:
29      app: grafana
30    ports:
31      - port: 3000
32        targetPort: 3000
33        nodePort: 30300

```

Useful commands:

Deploy Grafana:

kubectl apply -f grafana.yaml

Check Grafana pod:

kubectl get pods

Check Grafana service:

kubectl get svc

Access Grafana:

<http://<EC2-Public-IP>:30300>

Grafana improves observability by providing a centralized interface for monitoring and analyzing system behavior.

9.4 Monitoring Workflow

The monitoring workflow in AQI-Pulse is designed to continuously collect, store and visualize application metrics. The workflow begins when the application generates monitoring information through a metrics endpoint.

Prometheus periodically accesses the metrics endpoint and retrieves available data using its scraping mechanism. The collected information is stored and later used by Grafana for visualization.

Grafana queries Prometheus and displays monitoring data through dashboards containing charts and performance panels. Administrators can use these dashboards to monitor system behavior and identify potential issues.

The monitoring workflow followed in the project is:

AQI Application → Metrics Endpoint → Prometheus → Metrics Storage → Grafana → Dashboard → Users

Benefits of the monitoring workflow:

- Real-time metrics collection
- Continuous performance monitoring
- Dashboard-based visualization
- Improved observability
- Faster issue detection
- Better system analysis

Chapter 10: AWS Cloud Deployment

10.1 EC2 Infrastructure

Amazon Elastic Compute Cloud (EC2) is a cloud computing service provided by Amazon Web Services that allows users to create and manage virtual servers in the cloud. In the AQI-Pulse project, an AWS EC2 Ubuntu instance is used as the deployment environment for hosting Kubernetes resources and monitoring services.

The EC2 instance hosts:

- Kubernetes K3s Cluster
- AQI Application Pod
- Prometheus Pod
- Grafana Pod
- Jenkins CI/CD Environment

Deployment configuration:

Component	Configuration
Cloud Platform	AWS
Service	EC2
Instance Type	m7i-flex.large
Kubernetes	K3s
Cluster Type	Single Node
Replicas	1
AQI Pod	1
Prometheus Pod	1
Grafana Pod	1

☐	Name 	Instance ID	Instance state 	Instance type 	Status check	Availability Zone 	Public IPv4 DNS 
☐	aqi-node	i-00a6856b82e9e5876	 Running  	m7i-flex.large	 3/3 checks passed	ap-south-1a	ec2-13-235-254-240.ap...

Fig 10: Successful EC2 Instance

10.2 Security Group Configuration

AWS Security Groups act as virtual firewalls and control incoming and outgoing traffic for EC2 instances. Proper configuration of security groups is necessary to ensure external access to deployed services.

The following ports were configured:

Service	Port
SSH	22
Jenkins	8080
AQI Application	30001
Prometheus	30090
Grafana	30300

These rules allow users and administrators to access deployment and monitoring services securely.

10.3 Deployment Workflow

The deployment workflow in AQI-Pulse follows a cloud-native DevOps approach where automation tools and cloud infrastructure work together.

The deployment process begins when the developer pushes source code changes to GitHub. A GitHub webhook triggers Jenkins, which starts the CI/CD pipeline. Jenkins builds a Docker image and pushes it to Docker Hub.

The Kubernetes cluster running inside the EC2 instance pulls the latest image and creates deployment resources. Application services are exposed through Node-Port configuration.

Deployment workflow:

Developer → GitHub → Jenkins → Docker Image → Docker Hub → AWS EC2 → Kubernetes → Pods → Users

Useful commands:

Deploy resources:

```
kubectl apply -f aqi-deployment.yaml
```

```
kubectl apply -f aqi-service.yaml
```

Check resources:

```
kubectl get all
```

Chapter 11: Testing and Validation

11.1 Functional Testing

Functional testing is performed to verify whether the AQI-Pulse application behaves according to expected requirements. The purpose of this testing is to ensure that major application functionalities operate correctly and provide desired outputs.

The AQI-Pulse system was tested after deployment to validate dashboard accessibility, application behavior and user interaction capabilities.

The following functional test cases were performed:

Test Case	Expected Result	Status
Application launch	Dashboard opens successfully	Pass
Browser access	Application accessible through URL	Pass
AQI dashboard loading	Interface loads correctly	Pass
Kubernetes service access	Application reachable via Node-Port	Pass
User interaction	System responds correctly	Pass

The test results indicate that the application functionality operates successfully and fulfills project requirements.

11.2 Kubernetes Validation

Kubernetes validation was performed to verify whether deployment resources were created correctly and whether orchestration features functioned properly.

The AQI-Pulse application was successfully deployed in a single-node K3s cluster. Kubernetes created deployment resources and automatically generated ReplicaSets and Pods.

Validation checks performed:

- Deployment created successfully
- Pod entered Running state
- ReplicaSet created successfully
- Service exposed through Node-Port
- Self-healing behavior validated

Self-healing functionality was tested by manually deleting the running application pod. Kubernetes automatically created a new pod and restored application availability.

The validation process confirmed proper functioning of Kubernetes orchestration mechanisms.

```
ubuntu@aqi-node:~$ sudo kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
aqi-pulse-7694586d78-x5cr1         1/1     Running   5 (7h28m ago)  2d5h
grafana-5fddfc6dd-hqf9m           1/1     Running   4 (7h28m ago)  2d4h
prometheus-6cd9f585c9-jmpv4       1/1     Running   4 (7h28m ago)  2d3h
ubuntu@aqi-node:~$
```

Fig 11: Kubernetes pods running

11.3 Monitoring Validation

Monitoring validation was performed to verify whether Prometheus and Grafana successfully collected and visualized application metrics.

Prometheus periodically scraped metrics and stored monitoring information. Grafana successfully connected with Prometheus and displayed dashboard visualizations.

Validation results:

- Prometheus service running successfully
- Grafana dashboard accessible
- Metrics collected periodically
- Dashboard visualization working correctly

- Monitoring services integrated successfully

The monitoring system successfully provided visibility into application behavior and deployment status.

11.4 Deployment Validation

Deployment validation was performed to verify the complete DevOps workflow used in the project. The validation process included checking CI/CD functionality and application accessibility after deployment.

The deployment workflow executed successfully:

Developer → GitHub → Jenkins → Docker Hub → Kubernetes → Browser Access

Validation observations:

- Jenkins pipeline executed successfully
- Docker image uploaded to Docker Hub
- Kubernetes deployment completed successfully
- Application pod entered Running state
- Application accessible through Node-Port

The application was successfully accessed through:

`http://<EC2-Public-IP>:30001`

The successful deployment confirmed proper integration of GitHub, Jenkins, Docker, Kubernetes and AWS cloud infrastructure.

Chapter 12: Challenges and Solutions

During development and deployment of AQI-Pulse, multiple technical challenges were encountered while integrating cloud-native technologies and DevOps tools. Since the project combined Docker, Kubernetes, Jenkins, AWS, Prometheus and Grafana, resolving configuration and deployment issues required multiple iterations.

12.1 Jenkins Issues

During Jenkins pipeline setup, automatic build triggering was initially not functioning correctly. Even after pushing changes to GitHub, Jenkins builds were not starting automatically.

Issues faced:

- GitHub webhook trigger not working
- Jenkins not detecting repository updates
- Build pipeline not executing automatically

Solution implemented:

The issue was resolved by properly configuring webhook settings and enabling build trigger options inside Jenkins. The webhook URL was configured correctly and repository integration was verified.

Result:

Jenkins successfully triggered builds automatically after code updates.

12.2 Kubernetes Issues

During Kubernetes deployment, configuration-related issues were encountered while managing pods and services.

Issues faced:

- Pods entering unexpected states

- Service accessibility problems
- Node-Port configuration confusion
- Difficulty understanding ReplicaSet behavior

Solution implemented:

Deployment and service YAML files were corrected and Kubernetes resources were verified using cluster commands. Service mappings and labels were reconfigured appropriately.

Result:

Pods entered running state successfully and applications became accessible through Node-Port.

12.3 Monitoring Issues

Prometheus and Grafana integration initially presented monitoring configuration challenges.

Issues faced:

- Prometheus unable to scrape metrics
- Incorrect metrics target configuration
- Dashboard connection issues in Grafana

Solution implemented:

Prometheus configuration files were modified and target endpoints were corrected. Grafana datasource configuration was updated to establish proper communication with Prometheus.

Result:

Metrics collection and dashboard visualization started functioning successfully.

The challenges encountered during project implementation provided practical exposure to debugging deployment systems and understanding real-world DevOps workflows.

Chapter 13: Results and Discussion

The AQI-Pulse project was successfully developed and deployed using a cloud-native DevOps architecture. The implementation demonstrated successful integration of application development, containerization, CI/CD automation, orchestration and monitoring technologies.

The application deployment workflow operated successfully across all integrated components including GitHub, Jenkins, Docker, Kubernetes, AWS EC2, Prometheus and Grafana.

The observed implementation results are summarized below:

Component	Result
Streamlit Application	Successfully deployed
Docker Containerization	Successful
Jenkins Pipeline Execution	Successful
Docker Hub Image Push	Successful
Kubernetes Deployment	Successful
Pod Creation	Successful
Node-Port Access	Successful
Prometheus Monitoring	Successful
Grafana Dashboard	Successful

The Kubernetes cluster successfully created and managed application resources including Deployments, ReplicaSets and Pods. Self-healing functionality was validated by deleting application pods and observing automatic pod recreation.

Monitoring services successfully collected and visualized metrics through dashboards. Prometheus continuously scraped monitoring information while Grafana displayed performance dashboards.

The deployment architecture also demonstrated automation capability where application updates followed the workflow:

Developer → GitHub → Jenkins → Docker Hub → Kubernetes → Deployment

The implementation results indicate that the proposed system achieved its objectives related to automation, portability, scalability and observability.

Chapter 14: Advantages of System

The proposed AQI-Pulse system integrates cloud-native technologies and DevOps practices to provide a scalable and automated deployment environment. Compared to traditional deployment approaches, the system offers several advantages in terms of portability, monitoring and infrastructure management.

The major advantages of the system are:

- **Automated Deployment:**
Jenkins CI/CD pipeline reduces manual deployment effort and automates build and deployment activities.
- **Containerized Environment:**
Docker packages the application and dependencies together, eliminating dependency conflicts.
- **Cloud-Based Accessibility:**
Deployment on AWS EC2 allows users to access the application remotely.
- **Kubernetes Orchestration:**
Kubernetes manages deployments, pods and services automatically.
- **Self-Healing Capability:**
Failed application pods are recreated automatically by Kubernetes.
- **Improved Portability:**
Containers ensure that the application behaves consistently across environments.
- **Scalability Support:**
Kubernetes architecture supports future expansion and scaling requirements.
- **Real-Time Monitoring:**
Prometheus and Grafana provide monitoring and dashboard visualization.
- **Better Resource Management:**
Cloud infrastructure enables efficient utilization of computing resources.
- **Reduced Manual Errors:**
Automation reduces repetitive tasks and deployment mistakes.

These advantages improve deployment efficiency and demonstrate practical implementation of modern DevOps methodologies.

Chapter 15: Limitations

Although the proposed AQI-Pulse system successfully demonstrates cloud-native deployment and DevOps integration, certain limitations still exist in the current implementation. These limitations mainly arise due to infrastructure constraints and project scope boundaries.

The identified limitations of the system are as follows:

- **Single-Node Kubernetes Cluster:**
The project currently uses a single-node K3s cluster, which limits high availability and distributed deployment capabilities.
- **Limited Scalability Testing:**
Large-scale workload and stress testing were not performed during implementation.
- **No Horizontal Pod Autoscaling:**
Automatic scaling of pods based on workload conditions has not been implemented.
- **No HTTPS or Ingress Configuration:**
The application currently uses Node-Port services and does not include advanced ingress configurations.
- **Basic Monitoring Setup:**
Monitoring is implemented using Prometheus and Grafana but alert management functionality is not integrated.
- **Dynamic Public IP Dependency:**
AWS EC2 public IP addresses may change after instance restart, requiring updated application URLs.
- **Limited Security Configuration:**
Advanced security mechanisms such as authentication, authorization and role-based access control are not fully implemented.

These limitations provide opportunities for future improvements and expansion of the system architecture.

Chapter 16: Future Scope

The AQI-Pulse project successfully demonstrates the implementation of a cloud-native air quality dashboard integrated with DevOps technologies. However, several enhancements can be incorporated in future versions to improve functionality, scalability and system capabilities.

The future scope of the project includes:

- **Multi-Node Kubernetes Cluster:**
The current implementation uses a single-node K3s cluster. Future deployment can use a multi-node architecture to improve fault tolerance and availability.
- **Horizontal Pod Autoscaling (HPA):**
Kubernetes autoscaling can be implemented to automatically increase or decrease pod count based on workload and resource utilization.
- **Ingress Controller Configuration:**
Instead of Node-Port services, Ingress configuration can be used for better traffic management and domain-based routing.
- **HTTPS Integration:**
SSL/TLS certificates can be configured to provide secure communication.
- **Alert Management:**
Integration of Alertmanager with Prometheus can generate automatic alerts during failures or abnormal system behavior.
- **Enhanced Monitoring:**
Additional monitoring tools and logging systems can be integrated for improved observability.
- **Static Domain Deployment:**
A custom domain can be configured instead of relying on changing EC2 public IP addresses.
- **Advanced AQI Prediction Features:**
More sophisticated machine learning algorithms and larger datasets can be incorporated to improve prediction capabilities.
- **Role-Based Authentication:**
User authentication and access control mechanisms can be added for better security.

These future enhancements can improve scalability, security and production readiness of the AQI-Pulse system.

Chapter 17: Conclusion

The project AQI-Pulse: Cloud-Based Air Quality Dashboard with DevOps was successfully designed and implemented using modern cloud-native technologies and DevOps practices. The project combined application development, containerization, CI/CD automation, orchestration and monitoring into a unified deployment architecture.

The application was developed using Python and Streamlit and containerized using Docker to ensure portability and consistency across environments. Deployment automation was implemented using Jenkins, while Docker Hub was used for image management.

The application was deployed on an AWS EC2 instance configured with a single-node Kubernetes K3s cluster. Kubernetes successfully managed deployments, pods and services while demonstrating self-healing capability.

Monitoring functionality was implemented using Prometheus and Grafana, providing metrics collection and dashboard visualization for improved observability.

The deployment workflow:

Developer → GitHub → Jenkins → Docker Hub → Kubernetes → Users

operated successfully and demonstrated automation of software delivery processes.

The project successfully achieved its objectives related to:

- Cloud-based deployment
- Containerization
- CI/CD implementation
- Kubernetes orchestration
- Monitoring and observability
- Deployment automation

Overall, the project provided practical exposure to modern DevOps workflows and demonstrated how cloud-native technologies can be integrated to create scalable and manageable systems.

Chapter 18: References

The following references were used during development and implementation of the AQI-Pulse project. These resources provided guidance related to application development, containerization, cloud deployment, Kubernetes orchestration and monitoring technologies.

[1] Python Software Foundation, *Python Documentation*. Available: <https://docs.python.org/>

[2] Streamlit Documentation, *Build Data Applications in Python*. Available: <https://streamlit.io/>

[3] Docker Inc., *Docker Documentation*. Available: <https://docs.docker.com/>

[4] Jenkins Documentation, *Jenkins User Handbook*. Available: <https://www.jenkins.io/doc/>

[5] Kubernetes Documentation, *Production-Grade Container Orchestration*. Available: <https://kubernetes.io/docs/>

[6] K3s Documentation, *Lightweight Kubernetes*. Available: <https://docs.k3s.io/>

[7] Amazon Web Services Documentation, *Amazon EC2 User Guide*. Available: <https://docs.aws.amazon.com/ec2/>

[8] Prometheus Documentation, *Monitoring Systems and Services*. Available: <https://prometheus.io/docs/>

[9] Grafana Documentation, *Grafana Dashboard Documentation*. Available: <https://grafana.com/docs/>

[10] GitHub Documentation, *Version Control and Repository Management*. Available: <https://docs.github.com/>

[11] Red Hat, *Containerization and Kubernetes Concepts*, Red Hat Developer Documentation.

[12] E. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, *Borg, Omega and Kubernetes*, Communications of ACM.

[13] Martin Fowler, *Continuous Integration Concepts and Practices*.

Appendix

IMPORTANT COMMANDS REFERENCE

Docker Commands

Command	Purpose
<code>docker build -t archi2402/aqi-pulse:latest .</code>	Build Docker image
<code>docker images</code>	View available Docker images
<code>docker run -p 8501:8501 archi2402/aqi-pulse:latest</code>	Run Docker container
<code>docker ps</code>	View running containers
<code>docker ps -a</code>	View all containers
<code>docker stop <container-id></code>	Stop running container
<code>docker push archi2402/aqi-pulse:latest</code>	Upload image to Docker Hub
<code>docker pull archi2402/aqi-pulse:latest</code>	Pull image from Docker Hub
<code>docker login</code>	Authenticate Docker Hub

Kubernetes Commands

Command	Purpose
<code>kubectl apply -f aqi-deployment.yaml</code>	Deploy application
<code>kubectl apply -f aqi-service.yaml</code>	Deploy service
<code>kubectl get pods</code>	View running pods
<code>kubectl get deployments</code>	View deployments

Command	Purpose
kubectl get svc	View services
kubectl get nodes	View cluster nodes
kubectl get all	View all Kubernetes resources
kubectl describe pod <pod-name>	Detailed pod information
kubectl logs <pod-name>	View pod logs
kubectl delete pod <pod-name>	Delete pod manually
kubectl get pods -w	Watch pod recreation in real time
kubectl rollout status deployment/aqi-pulse	Check deployment status

AWS Commands

Command	Purpose
curl ifconfig.me	Get EC2 public IP
hostname -I	View local IP address
free -h	View memory usage
lscpu	View CPU information
df -h	View disk utilization
sudo netstat -tulnp	View open ports
sudo ss -tulnp	Display active listening ports
top	Monitor running processes
htop	Interactive system monitoring
uptime	View server uptime